



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Laboratorio 3 Automatización de un proceso con n8n

Estudiante: Camilo Rivano Lara
Rol: 202304534-9
Paralelo: 101
Profesor: José Luis Sáez Tamayo

Repositorio: <https://github.com/camilorivano/ICN292-Lab3-Rivano-Camilo>

Resumen Ejecutivo

El objetivo de este informe es presentar el trabajo del laboratorio 3 de la asignatura Sistemas de la información para la gestión, explicando qué se construyó, cómo se probó, qué se observó al ejecutarlo y qué se concluye del resultado.

El caso sigue siendo AndesHogar SpA. Las solicitudes de devolución llegan desde el formulario del sitio web a un buzón compartido y una persona las lee una por una para decidir si se aprueban, se rechazan o se mandan a revisión. Ese trabajo demora en promedio tres días, cuando la política que la empresa le promete al cliente es de veinticuatro horas. El cuello de botella no está en la inspección física del producto, que necesariamente la hace una persona, sino en la clasificación previa: leer la solicitud, compararla contra la política y decidir un destino.

Lo que se construyó es un flujo en n8n que hace esa clasificación sola. Recibe cada solicitud por un webhook, la evalúa contra seis reglas en un orden fijo y la deriva a una de cuatro rutas: aprobación, rechazo, revisión humana o datos inválidos. Cada decisión queda registrada con su identificador, su SKU, su monto, la ruta, el motivo y los umbrales con que se evaluó, de manera que después se pueda auditar. Un segundo flujo corre todos los días a una hora fija, lee lo del día y arma un resumen con la distribución por ruta, los montos y la tasa de aprobación automática.

Con las quince solicitudes entregadas el flujo clasificó cinco como aprobación, cuatro como rechazo y seis como revisión, por un total de \$1.251.820 y una tasa de aprobación automática de 33,3%. El dato que importa para la operación no es esa cifra sino su complemento: nueve de quince quedaron resueltas sin que nadie las mirara, así que la persona a cargo pasa de revisar quince casos a revisar seis.

El resultado es un producto mínimo viable y no un sistema definitivo. Funciona bien con entradas bien formadas, pero las pruebas de la parte C mostraron tres limitaciones que hay que resolver antes de operarlo con volumen real, y que se desarrollan en la parte D.

La semilla utilizada en este caso fue de $S = 249$

Los parámetros del caso fueron

Los parámetros del caso fueron

S	= tres últimos dígitos del RUT	-> 249
U	= $30000 + 1000 * \text{RESIDUO}(S; 50)$	-> \$79.000
D	= $7 + 7 * \text{RESIDUO}(S; 4)$	-> 14 días

Con $D = 14$ días, con $U = \$79.000$ Los tres casos de control que venían en el archivo, DEV-C01, DEV-C02 y DEV-C03, sirven para verificar indirectamente que los parámetros están bien calculados, porque su ruta no depende de S. Los tres cayeron donde correspondía, así que los umbrales aplicados son los correctos.

Parte A – Flujo de triage

El flujo principal tiene once nodos y se puede leer en tres tramos: la entrada y la clasificación, la ramificación por destino, y un tramo final que es común a las cuatro rutas.

Webhook -> Switch -> 4 x Edit Fields -> UNIFICAR -> UF_INDICADOR ->

REGISTRO -> GUARDAR_REGISTRO -> NOTIFICACION -> RESPUESTA

El Webhook expone una dirección que escucha peticiones POST en la ruta /triage-devoluciones. Se configuró con el parámetro Respond en “Using Respond to Webhook Node” para que la respuesta al cliente se arme al final del flujo y no apenas llega la solicitud. El Switch, en modo Rules, es donde vive toda la política de negocio. Los cuatro nodos Edit Fields escriben la ruta y el motivo. El tramo final consulta el valor de la UF, arma el registro, lo guarda, compone la notificación y responde.

Las reglas y el orden en que se evalúan

Las seis reglas se evalúan de arriba hacia abajo y la primera que se cumple detiene la evaluación. El orden no es un detalle de implementación, es parte de la política.

N°	Condición	Tipo de dato	Salida
1	id_solicitud está vacío	String	INVALIDO_CAMPO
2	monto.toNumber() ≤ 0	Number	INVALIDO_MONTO
3	dias_desde_compra.toNumber() > 14	Number	RECHAZO_PLAZO
4	estado_producto = danado_por_uso	String	RECHAZO_ESTADO
5	monto.toNumber() > 79000	Number	REVISION_MONTO
6	estado_producto = con_fallas	String	REVISION_FALLA
—	Ninguna de las anteriores (Fallback Output)		— APROBACION

El efecto del orden se puede ver en los datos reales. La solicitud DEV-1119 tiene veinticinco días desde la compra y además el producto está dañado por uso, o sea cumple la regla 3 y la regla 4 al mismo tiempo. Como la regla 3 se evalúa primero, sale por RECHAZO_PLAZO y el motivo que queda registrado es fuera_de_plazo y no danado_por_uso. Lo mismo pasa con DEV-1120, que cumple la 5 y la 6 y sale por la primera. Si el orden estuviera invertido esas mismas solicitudes tomarían rutas distintas con reglas idénticas, y por eso el registro guarda el motivo además de la ruta.

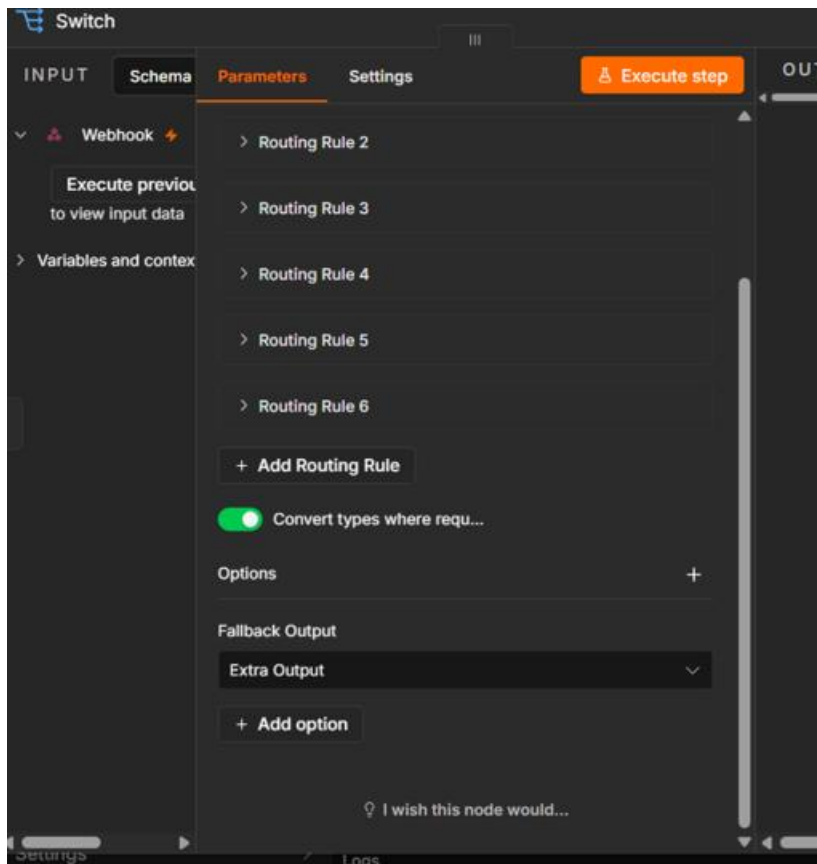


Figura 1 — Nodo Switch con las seis reglas y el Fallback Output en “Extra Output”

Decisiones de diseño

El enunciado dice qué tiene que hacer el flujo, pero no cómo cablearlo, así que hubo que tomar algunas decisiones durante la construcción. Se dejan declaradas para que se puedan evaluar.

Decisión Por qué se tomó

Se agregó un nodo “No Operation” llamado UNIFICAR donde convergen las cuatro ramas. El nodo HTTP Request reemplaza el ítem de entrada con la respuesta del servicio, así que después de consultar la UF ya no están la ruta ni el motivo, que son campos que la regla 7 exige registrar. Con un punto común se pueden recuperar con `$('UNIFICAR').item.json.campo`.

Los cuatro Edit Fields determinan el motivo con una condición en vez de texto fijo. Tres de ellos reciben dos causales distintas, así que el motivo no se puede escribir fijo. La condición replica el orden de evaluación del Switch. El costo de esta decisión se verificó como riesgo real en la parte C.

El registro se guarda en una tabla de datos nativa de n8n llamada `registro_devoluciones`. El enunciado pide que el flujo programado lea el registro pero no dice dónde guardarlo. Se descartó Google Sheets porque necesita credenciales externas, y la consulta al historial de ejecuciones porque necesita una clave de API.

La notificación se compone pero no se despacha. Enviar correo de verdad exige credenciales SMTP, y el enunciado pide que el archivo entregado no tenga contraseñas ni tokens. Conectar un proveedor sería un nodo más que no cambia la lógica de clasificación.

Tabla 2 — Decisiones de diseño tomadas durante la implementación

Servicio externo consultado

El flujo consulta el servicio público <https://mindicador.cl/api> con una petición GET sin credenciales. De la respuesta se consume el campo uf.valor, que se guarda en la columna uf_valor y se usa para calcular el monto de cada solicitud expresado en unidades de fomento.

La conversión tiene sentido para el negocio. Un umbral fijo en pesos va perdiendo significado con los años por la inflación, mientras que el mismo monto expresado en UF permite comparar una devolución de 2026 con una de años anteriores.

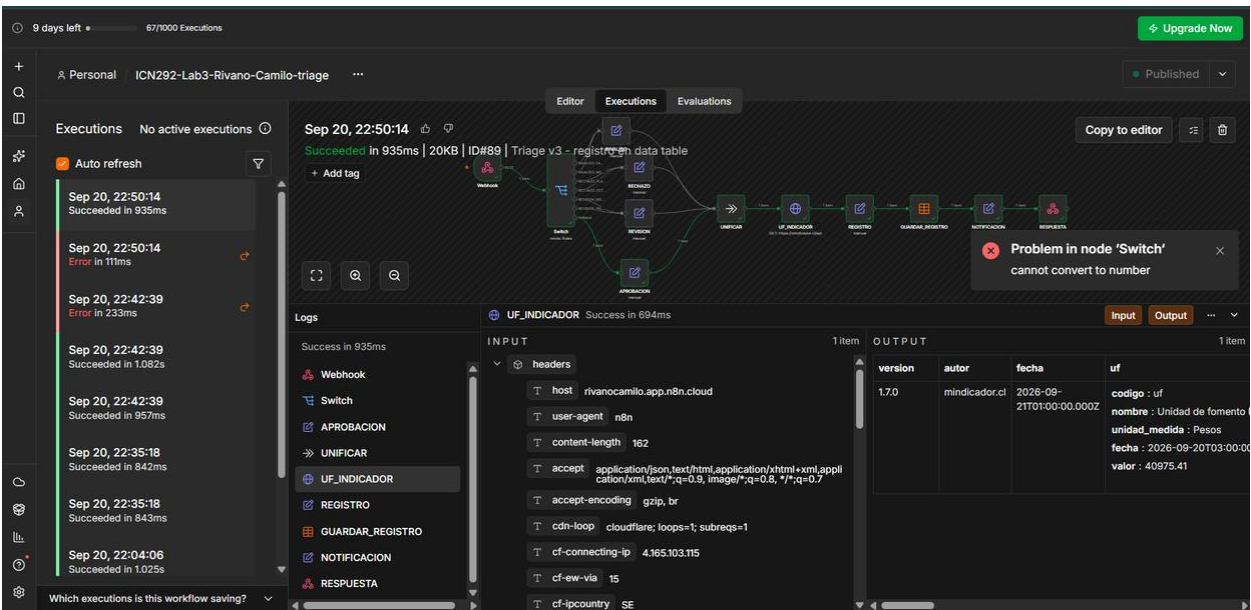


Figura 2 — Salida del nodo UF_INDICADOR con el campo uf.valor que consume el flujo

Resultados de las quince solicitudes

Antes de ejecutar se predijo la distribución aplicando las reglas a mano, y el resultado de la ejecución coincidió exactamente con lo predicho.

ID	SKU	Monto	Días	Estado producto	Ruta	Motivo
DEV-1115	AU-210	\$19.990	3	sin_abrir	APROBACION	dentro_de_politica
DEV-1116	TE-330	\$69.980	5	abierto_sin_uso	APROBACION	dentro_de_politica
DEV-1117	HV-050	\$24.990	12	con_fallas	REVISION	producto_con_fallas
DEV-1118	MW-150	\$79.990	4	sin_abrir	REVISION	monto_sobre_umbral
DEV-1119	PL-120	\$45.990	25	danado_por_uso	RECHAZO	fuera_de_plazo
DEV-1120	AS-260	\$89.990	2	con_fallas	REVISION	monto_sobre_umbral
DEV-1121	LV-330	\$289.990	9	sin_abrir	REVISION	monto_sobre_umbral

DEV-1122	PL-120	\$45.990	2	sin_abrir	APROBACION	dentro_de_politica
DEV-1123	AU-210	\$39.980	15	danado_por_uso	RECHAZO	fuera_de_plazo
DEV-1124	HV-050	\$49.980	1	abierto_sin_uso	APROBACION	dentro_de_politica
DEV-1125	TE-330	\$34.990	22	sin_abrir	RECHAZO	fuera_de_plazo
DEV-1126	MW-150	\$79.990	8	con_fallas	REVISION	monto_sobre_umbral
DEV-C01	CB-010	\$9.990	1	sin_abrir	APROBACION	dentro_de_politica
DEV-C02	LV-330	\$289.990	2	sin_abrir	REVISION	monto_sobre_umbral
DEV-C03	MW-150	\$79.990	29	abierto_sin_uso	RECHAZO	fuera_de_plaz

Tabla 3 — Clasificación de las quince solicitudes del archivo entregado

The screenshot shows a workflow execution interface with a dark theme. At the top, there's a 'Execute workflow' button. Below it, a 'Logs' section shows 'ENVIAR_A_TRIAGE' completed successfully in 4.491s. The main area is divided into 'INPUT' and 'OUTPUT' sections. The 'INPUT' section lists 15 items with fields like id_solicitud, sku, unidades, monto, dias_desde_compra, estado_producto, and email_cliente. The 'OUTPUT' section lists 15 items with fields id_solicitud, ruta, and motivo. The output data matches the data in Table 3.

id_solicitud	ruta	motivo
DEV-1115	APROBACION	dentro_de_politica
DEV-1116	APROBACION	dentro_de_politica
DEV-1117	REVISION	producto_con_fallas
DEV-1118	REVISION	monto_sobre_umbral
DEV-1119	RECHAZO	fuera_de_plazo
DEV-1120	REVISION	monto_sobre_umbral
DEV-1121	REVISION	monto_sobre_umbral
DEV-1122	APROBACION	dentro_de_politica
DEV-1123	RECHAZO	fuera_de_plazo
DEV-1124	APROBACION	dentro_de_politica
DEV-1125	RECHAZO	fuera_de_plazo
DEV-1126	REVISION	monto_sobre_umbral
DEV-C01	APROBACION	dentro_de_politica
DEV-C02	REVISION	monto_sobre_umbral
DEV-C03	RECHAZO	fuera_de_plazo

Figura 3 — Respuesta que devuelve el triage a las quince solicitudes enviadas

La distribución que resulta es la siguiente.

Ruta	Solicitudes	Monto total
APROBACION	5	\$195.930
RECHAZO	4	\$200.950
REVISION	6	\$854.940
INVALIDO	0	\$0
Total	15	\$1.251.820

Tabla 4 — Distribución por ruta y monto acumulado

Hay tres salidas del Switch que no recibieron ninguna solicitud. Las dos de datos inválidos quedaron vacías porque el archivo entregado no trae registros mal formados, lo cual era esperable porque el enunciado pide construirlos aparte. La salida RECHAZO_ESTADO también quedó vacía, pero en su caso por precedencia: las dos únicas solicitudes con producto dañado por uso, DEV-1119 y DEV-1123, estaban además fuera de plazo y las capturó la regla 3 antes.

También conviene mencionar que el archivo trae un valor de estado_producto que el enunciado no menciona, que es abierto_sin_uso y aparece en DEV-1116, DEV-1124 y DEV-C03. Ninguna regla lo considera, así que esas solicitudes se evalúan solo por monto y plazo. El comportamiento es correcto según la política, porque todo lo demás va a aprobación, pero conviene dejarlo escrito.

Parte B – Flujo programado de resumen

El flujo de resumen tiene ocho nodos. Un disparador programado inicia la ejecución, un nodo lee de la tabla las solicitudes del día, tres ramas paralelas calculan el desglose por ruta, el total del día y las aprobaciones automáticas, y un nodo final compone el mensaje.

Dos de esos nodos existen por cómo funciona la herramienta y no por la lógica del negocio. El nodo Aggregate junta en un solo ítem las tres líneas que produce el Summarize agrupado, porque en n8n cada nodo se ejecuta una vez por cada ítem que recibe y sin juntarlas se mandarían tres mensajes en lugar del mensaje único que pide el enunciado. El nodo Merge funciona como barrera de sincronización: n8n recorre las ramas una después de otra y sin esa barrera el nodo de mensaje se ejecutaba antes de que las otras dos ramas hubieran corrido, fallando con el error “Node TOTAL_DIA hasn't been executed”

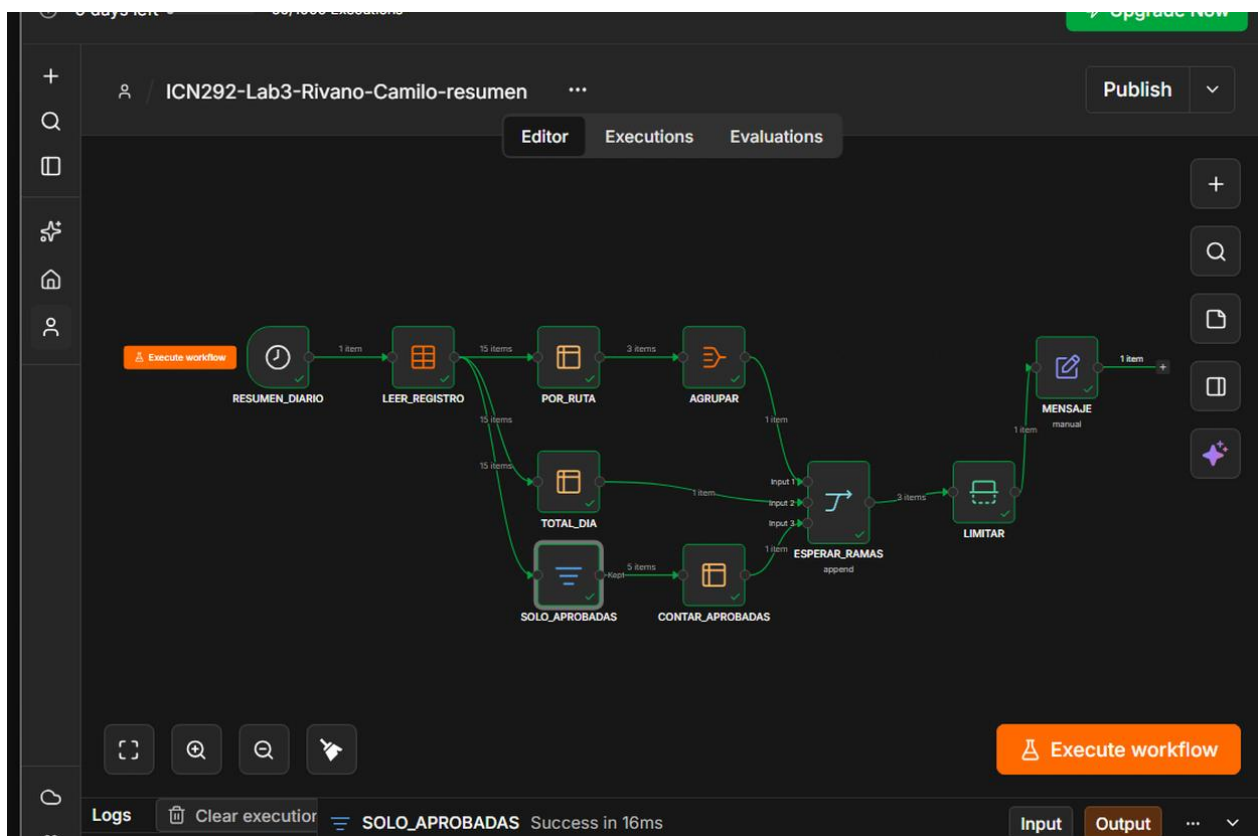


Figura 4 — Flujo de resumen con las tres ramas de cálculo convergiendo en la barrera de sincronización

El mensaje entrega los cuatro datos que pide el enunciado. Ejecutado sobre las quince solicitudes de prueba el resultado fue el siguiente.

Dato exigido	Valor obtenido
Solicitudes por ruta	APROBACION 5 · RECHAZO 4 · REVISION 6
Monto total por ruta	\$195.930 · \$200.950 · \$854.940
Monto total del día	\$1.251.820
Tasa de aprobación automática	33,33

Tabla 5 — Contenido del mensaje del resumen diario

La consistencia entre las dos ramas de cálculo se verificó sumando los montos por ruta, porque $\$195.930 + \$200.950 + \$854.940$ da $\$1.251.820$, que es el mismo total que calcula la otra rama de forma independiente.



fecha	total_solicitudes	monto_total_día	aprobadas_automaticas	tasa_aprobacion_pct
2026-09-20	15	1251820	5	33.33333333333333

Figura 5 — Salida del nodo MENSAJE con los cuatro datos del resumen

Frecuencia elegida

Se configuró una ejecución diaria a las 20:00 horas.

La política que la empresa le promete al cliente es de veinticuatro horas, así que el día es la unidad natural de control del proceso. Un resumen semanal llegaría demasiado tarde para corregir algo que se desvió el lunes, y uno cada hora produciría ruido sin información accionable. La hora responde al cierre de la jornada laboral, porque un resumen emitido a mediodía informaría sobre un día incompleto y obligaría a revisar dos veces lo mismo.

Qué ocurre en un día sin solicitudes

El nodo de lectura filtra las filas cuya columna fecha coincide con la fecha de ejecución, así que si ese día no hubo solicitudes devuelve cero ítems.

Esto se verificó sin querer al ejecutar el flujo un día después de haber poblado la tabla. El nodo LEER_REGISTRO terminó en verde con cero ítems y los nodos siguientes ni se ejecutaron. El flujo no falla ni avisa nada, simplemente no manda el mensaje.

Eso es un fallo silencioso. Desde el punto de vista de quien espera el resumen, un día sin solicitudes y un flujo que dejó de funcionar son exactamente lo mismo, porque en los dos casos no llega nada. La corrección adecuada es que el flujo mande igual un mensaje diciendo que el día cerró sin movimientos, lo que se consigue activando la opción “Always Output Data” en el nodo de lectura o agregando una rama explícita para ese caso.

Parte C – Errores, límites y casos borde

Tres entradas inválidas

Se construyeron tres solicitudes con defectos distintos y se enviaron al flujo publicado.

Carga enviada	Comportamiento esperado	Comportamiento observado
id_solicitud: ""	Regla 1, ruta INVALIDO	INVALIDO con motivo falta_id_solicitud. Responde 200 con la clasificación.
monto: "0"	Regla 2, ruta INVALIDO	INVALIDO con motivo monto_no_positivo. Responde 200 con la clasificación.
dias_desde_compra: "tres"	Falla del nodo Switch	El Switch aborta con el mensaje «cannot convert to number». El emisor recibe un error HTTP en vez de una clasificación.

Tabla 6 — Las tres entradas inválidas con su comportamiento esperado y observado

La diferencia entre los dos primeros casos y el tercero es lo importante de este punto. En los casos 1 y 2 el dato es inválido pero legible, o sea el flujo lo entiende, lo clasifica como inválido y le responde al cliente. En el caso 3 el dato no se puede ni siquiera evaluar, porque la regla 3 intenta convertir el texto “tres” a número con. toNumber() y el nodo se detiene ahí.

La consecuencia operativa es distinta en cada caso. Las dos primeras solicitudes reciben respuesta y quedan registradas, mientras que la tercera no deja ningún registro y su cliente se queda sin información. Además se verificó que la falla de esa solicitud no impidió procesar las otras dos, porque las tres ejecuciones aparecen en el historial del flujo, dos exitosas y una con error.

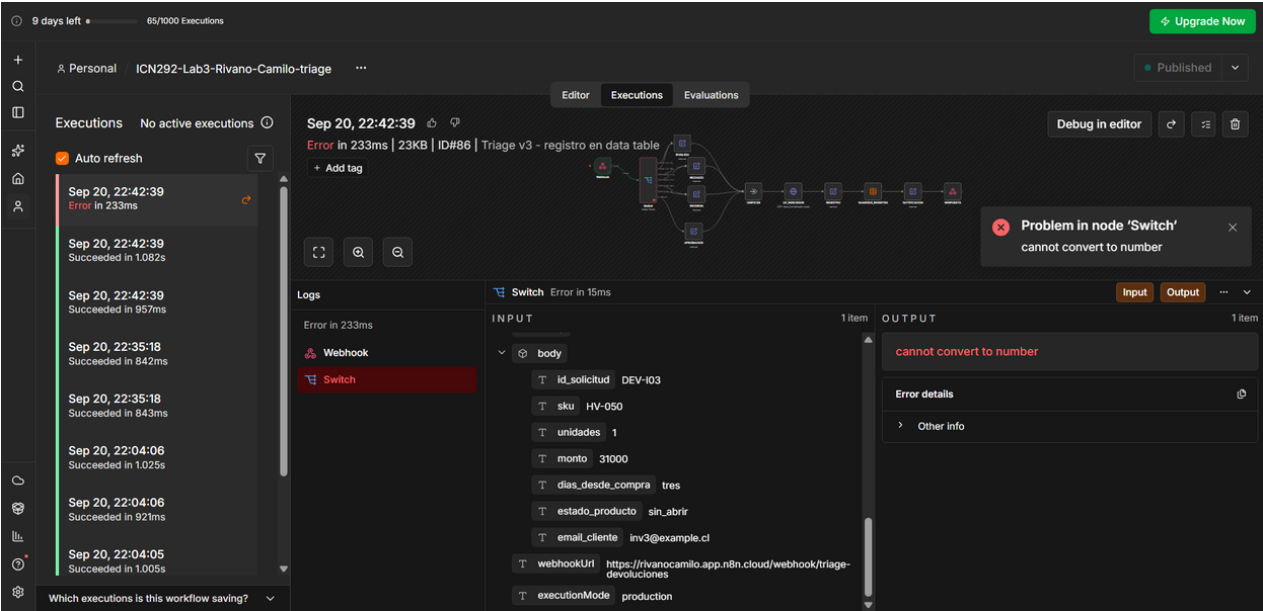


Figura 6 — El Switch aborta al intentar convertir el texto tres a número

Los tres valores de On Error

Cada nodo de n8n tiene en su pestaña Settings un parámetro On Error con tres valores posibles.

- Stop Workflow. Es el valor por defecto. El nodo falla, la ejecución se detiene y queda marcada en rojo en el historial.
- Continue (using regular output). El ítem sigue por la salida normal llevando una clave error adentro, y los nodos siguientes lo procesan sin distinguirlo de un ítem válido.
- Continue (using error output). El nodo gana una segunda salida por la que salen los ítems que fallaron, y se les puede conectar un camino propio.

En este flujo todos los nodos quedaron en Stop Workflow. La razón es que no se construyó ninguna rama que maneje errores, y poner “Continue” sin esa rama produce el escenario más peligroso de todos: el flujo termina en verde, el cliente recibe una respuesta de aprobación, y el registro nunca se escribió porque el nodo que debía escribirlo falló sin detener la ejecución. Un flujo que nunca falla no es un flujo que funciona.

Nodo	On Error	Razón
Webhook	Stop Workflow	Si no puede recibir la solicitud no hay nada que procesar.
Switch	Stop Workflow	Un dato que no se puede evaluar no debe clasificarse. Es mejor un error visible que una ruta asignada sobre un valor que el flujo no entendió.
Edit Fields (los cuatro)	Stop Workflow	Sin ruta ni motivo el registro no se puede auditar, lo que incumple la regla 7.
UNIFICAR	Stop Workflow	No ejecuta ninguna operación, así que no tiene manera de fallar.
UF_INDICADOR	Stop Workflow con Retry	Es el único nodo con Retry On Fail activo. Se detalla más abajo.
REGISTRO	Stop Workflow	Si no se puede armar el registro se pierde la trazabilidad de la decisión.
GUARDAR_REGISTRO	Stop Workflow	Es el caso crítico. Con «Continue» el cliente recibiría su aprobación sin que quedara constancia de ella en ninguna parte.
NOTIFICACION	Stop Workflow	El mensaje es parte del compromiso con el cliente.
RESPUESTA	Stop Workflow	Sin respuesta el emisor no sabe qué se decidió.

Tabla 7 — Valor de On Error elegido en cada nodo y su justificacióm

En el nodo UF_INDICADOR se activó además Retry On Fail con tres intentos y un segundo de espera entre ellos, y se le puso un Timeout de diez segundos. Es el único nodo que depende de un servicio externo y por lo mismo el único cuya falla puede ser transitoria, porque una caída momentánea de red se resuelve reintentando. Reintentar no sirve para datos inválidos, ya que un monto que dice “veinte mil” falla igual las tres veces.

El Timeout responde a una observación concreta. Durante las pruebas una ejecución tardó un minuto y dieciocho segundos frente al segundo que tardaba normalmente, por lentitud del servicio consultado, y durante todo ese rato la respuesta al cliente quedó retenida. Acotar la espera permite fallar rápido y visible en vez de dejar al cliente esperando sin saber qué pasa.

Error Workflow y el fallo silencioso

Un Error Workflow es un flujo aparte que empieza con un nodo Error Trigger y que n8n ejecuta solo cuando otro flujo falla. Recibe el detalle de la ejecución fallida, o sea qué flujo, qué nodo, qué mensaje de error y a qué hora, y con eso puede avisar por correo, escribir en una tabla o abrir un ticket. Se asigna en Workflow settings eligiendo cuál se dispara cuando el flujo falle.

La necesidad viene de que a los flujos automatizados no los está mirando nadie. Un flujo disparado por webhook o por horario no tiene una persona esperando su resultado, así que si falla el error queda anotado en la pestaña Executions y ahí se queda hasta que alguien decida entrar a revisar.

Durante este laboratorio se observaron tres formas distintas de fallar en silencio, todas verificadas:

- El flujo programado que no está publicado. El Schedule Trigger solo corre si el workflow está activo, y uno inactivo no produce ningún error, simplemente no pasa nada y nadie recibe el resumen.
- El nodo que recibe cero ítems. Verificado al ejecutar el resumen sobre una fecha sin registros: el nodo de lectura terminó en verde con cero ítems y los siguientes no se ejecutaron.
- La condición que nunca se cumple. En el nodo Filter del flujo de resumen, un espacio en blanco al final del valor de comparación hacía que ninguna solicitud calzara con la ruta APROBACION. El flujo

terminaba en verde y la tasa de aprobación se reportaba como cero. Se detectó reemplazando temporalmente el operador de igualdad por “contains”, que sí devolvió resultados.

En los tres casos la ejecución termina correctamente y no hay error que un Error Workflow pueda capturar, porque técnicamente no falló nada: el flujo hizo exactamente lo que se le pidió, sobre un conjunto vacío o con una condición imposible de cumplir.

De ahí sale una distinción útil. El Error Workflow resuelve el fallo ruidoso, avisando cuando un nodo aborta, pero no resuelve el silencioso, donde el flujo termina bien sin producir nada o produciendo algo incorrecto. Para ese segundo caso hace falta verificar el resultado y no el estado, por ejemplo que el resumen avise explícitamente cuando el día cerró sin solicitudes en vez de terminar sin mandar nada.

Los límites: las comparaciones inclusivas

Las quince solicitudes entregadas no traen ningún caso de frontera, así que se construyeron dos. Los valores de relleno se eligieron a propósito para aislar la variable que se quería probar en cada caso: DEV-B01 tiene solo tres días desde la compra y el producto sin abrir, para que ninguna otra regla se active y el resultado dependa únicamente del umbral de monto, y DEV-B02 tiene un monto muy por debajo del umbral por la razón inversa.

Solicitud	Monto	Días	Regla que se evalúa	Resultado
DEV-B01	\$79.000	3	Regla 5: $79000 > 79000 \rightarrow$ falso	APROBACION
DEV-B02	\$30.000	14	Regla 3: $14 > 14 \rightarrow$ falso	APROBACION

Tabla 8 — Las dos solicitudes de frontera construidas y su resultado

Las dos fueron aprobadas, lo que confirma que los umbrales pertenecen a la zona aceptada: un monto igual a U está aprobado y un plazo igual a D está dentro de plazo. La distinción tiene consecuencia comercial directa, porque una devolución de exactamente \$79.000 se resuelve automáticamente en vez de ir a revisión, y es el tipo de decisión que conviene dejar escrita porque no se deduce de la implementación sin ir a leer el operador.

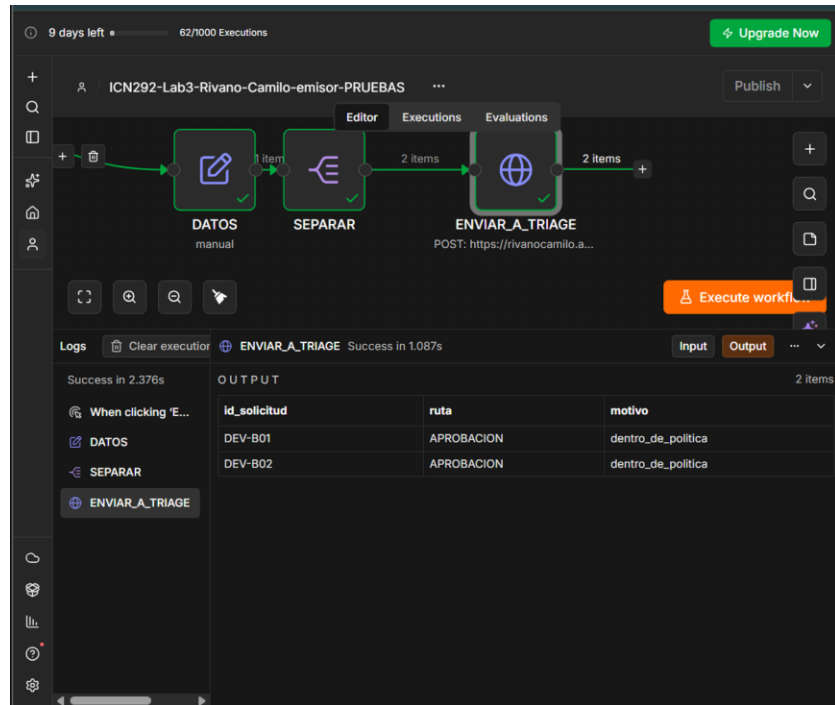


Figura 7 — Las dos solicitudes de frontera clasificadas como APROBACION

El error de tipo hecho a propósito

Se modificó a propósito la regla 5 del Switch para ver qué pasa cuando se compara con el tipo de dato equivocado, enviando la solicitud de control DEV-C01, de \$9.990, que en condiciones normales se clasifica como aprobación.

La versión de n8n que se usó no permite reproducir directamente el experimento que muestra el material del curso, porque al seleccionar el tipo String el desplegable de operadores no ofrece comparaciones de orden, solo igualdad, pertenencia y expresiones regulares. Por eso se hicieron dos pruebas complementarias.

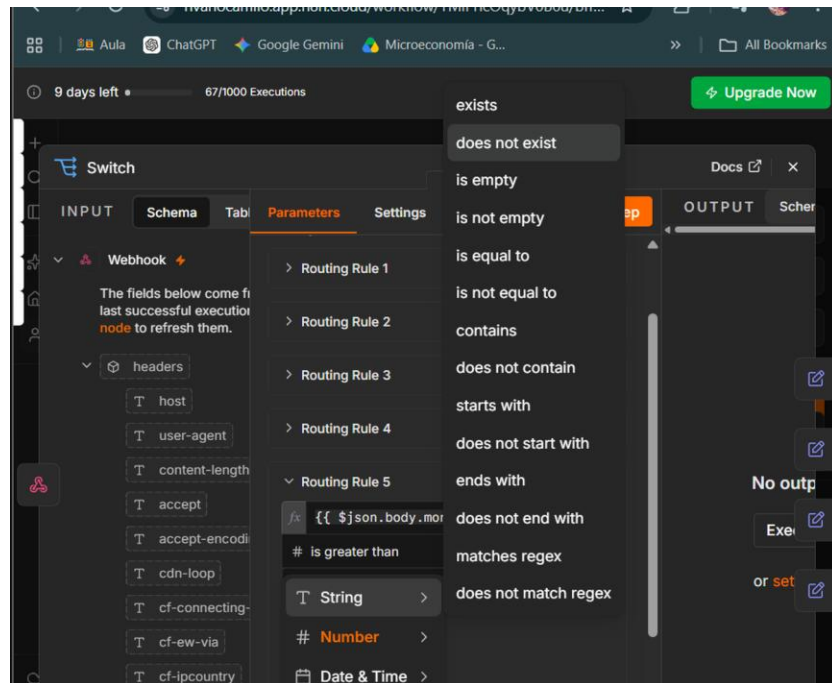
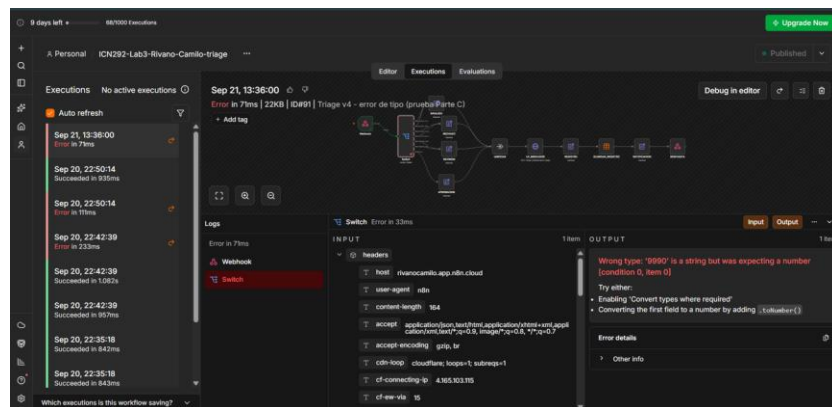


Figura 8 — Operadores disponibles para el tipo String: no incluyen comparaciones de orden

Prueba A. Se reemplazó la expresión por `$json.body.monto` sin `.toNumber()`, manteniendo el tipo Number y desactivando la opción “Convert types where required”. El resultado fue un error explícito que dice “Wrong type: '9990' is a string but was expecting a number”, y la misma herramienta sugiere las dos correcciones posibles, que son activar la conversión automática o agregar `.toNumber()`. O sea que n8n valida el tipo antes de comparar y detiene la ejecución, que es justamente la alternativa que el material del curso descarta.



Prueba B. Para obtener la ruta que el enunciado pide mostrar, se reescribió la condición como una expresión booleana que compara dos cadenas, `$json.body.monto > "79000"`, con el operador “is true”. Ahí sí el resultado fue el esperado y DEV-C01 se clasificó como REVISION.

La causa es que la comparación de textos se hace carácter por carácter siguiendo el orden alfabético. Al comparar “9990” con “79000” el primer carácter de la izquierda es 9 y el de la derecha es 7, y como el 9 va después del 7 la comparación da verdadero, de modo que una devolución de nueve mil novecientos noventa pesos se trata como si superara los setenta y nueve mil.

Hay un hallazgo adicional que vale la pena dejar anotado. El registro que quedó era incorrecto en dos dimensiones y no en una. La ruta asignada fue REVISION, como se esperaba, pero el motivo registrado fue producto_con_fallas siendo que DEV-C01 tiene el producto sin abrir. La explicación está en el acoplamiento que se mencionó en la parte A: el nodo Edit Fields de la rama de revisión determina el motivo con una condición propia que sí convierte el monto a número, así que al modificar solamente la regla del Switch las dos condiciones quedaron desincronizadas. El Switch derivó la solicitud por monto alto mientras el Edit Fields concluyó, correctamente según su propia lógica, que el monto no superaba el umbral y le atribuyó la causal al estado del producto por descarte.

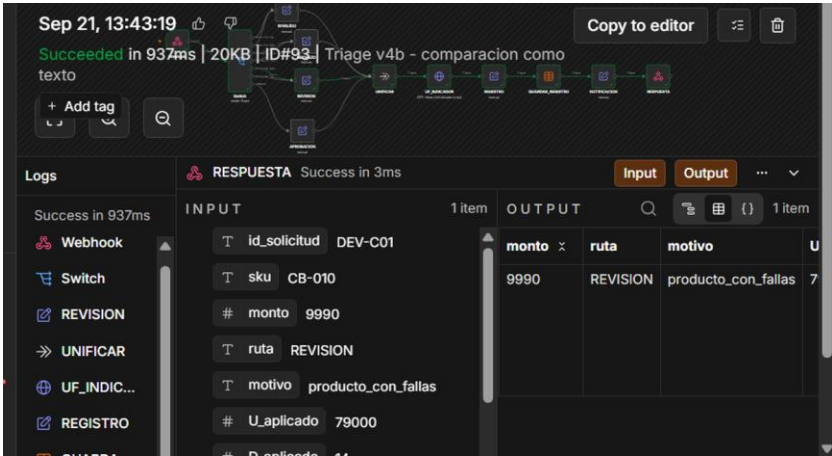


Figura 10 — DEV-C01, de \$9.990, clasificada como REVISION y con un motivo que no corresponde

Un solo cambio produjo entonces una ruta equivocada y un motivo falso, sin generar ningún error. Esto confirma el riesgo del diseño de cuatro nodos con motivo calculado, porque la corrección del registro depende de que dos lugares del flujo se mantengan sincronizados y nada en la herramienta lo verifica. La alternativa sería un nodo Edit Fields por cada salida del Switch con el motivo escrito como texto fijo, que elimina el acoplamiento a cambio de tres nodos más.

El defecto se detectaría comparando el registro contra el dato de origen, porque una solicitud clasificada por monto alto cuyo monto registrado es inferior al umbral es una contradicción por construcción. Una verificación automática de esa consistencia lo pillaría sin depender de que alguien revise caso a caso. Terminadas las dos pruebas la regla se restauró a su forma original y se verificó, reenviando DEV-C01, que vuelve a clasificarse como APROBACION con motivo dentro_de_politica.

Los dos casos que el flujo no puede resolver

Caso enviado	Qué pasa	Qué necesitaría el flujo para detectarlo
DEV-X01 con monto “veinte mil”	El Switch intenta convertir ese texto a número en la regla 2 y el nodo aborta con “cannot convert to number”. El flujo no alcanza a clasificar la solicitud y el cliente recibe un error HTTP en vez de una decisión.	Una etapa de validación antes del Switch que verifique que los campos numéricos son convertibles y derive los que no lo son a la ruta de datos inválidos, en lugar de dejar que la conversión falle dentro de la lógica de negocio.
DEV-X02 con SKU “ZZ-999”	Ninguna de las seis reglas consulta el SKU, así que el flujo lo ignora y clasifica la solicitud por monto, plazo y estado, resultando APROBACION sin error ni advertencia. El sistema aprueba la devolución de un producto que la empresa nunca vendió.	Consultar un catálogo de productos, sea una tabla de datos o un servicio, y rechazar los SKU no reconocidos antes de aplicar la política de devolución.

Tabla 9 — Los dos casos irresolubles con su causa y la corrección que requerirían

El contraste entre los dos casos es lo interesante. La primera falla de forma ruidosa, o sea se detiene, deja rastro y obliga a intervenir. La segunda falla en silencio y produce una decisión incorrecta que nadie va a cuestionar, porque desde el punto de vista del sistema todo funcionó bien. El segundo es por eso el más peligroso de los dos.

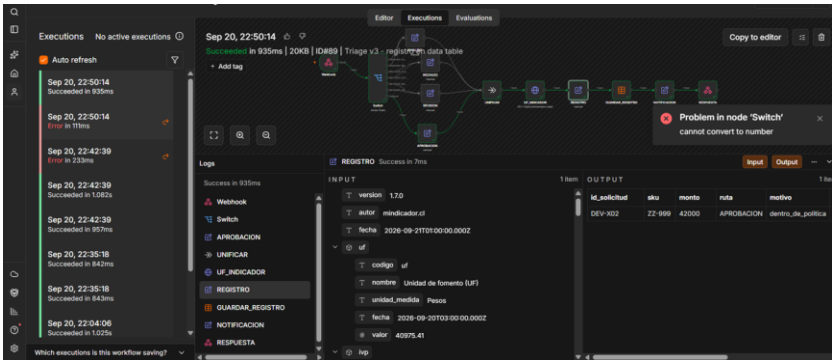


Figura 11 — DEV-X02 con SKU ZZ-999 registrada como APROBACION, sin error ni advertencia

Parte D – De producto mínimo viable a sistema definitivo

La pregunta de cierre es si este flujo debería convertirse en el sistema definitivo de AndesHogar. La respuesta es que no en el estado en que está, pero sí sirve como base.

Lo que el flujo resuelve bien está demostrado. Clasifica correctamente las quince solicitudes según la política, deja registro auditable de cada decisión con su motivo y sus umbrales, responde al cliente en segundos en vez de días, y baja de quince a seis los casos que necesitan atención humana. Eso es exactamente lo que el proyecto se propuso hacer.

Lo que impide operarlo con volumen real son tres carencias que aparecieron en las pruebas.

Carencia	Consecuencia	Corrección propuesta
No valida el formato de los datos que recibe	Un campo numérico mal escrito detiene el flujo y deja al cliente sin respuesta	Etapas de validación previa al Switch que derive los datos no convertibles a la ruta de inválidos
No verifica que el SKU exista en el catálogo	Puede aprobar devoluciones de productos que la empresa nunca vendió, sin error ni alerta	Consulta a un catálogo de productos antes de aplicar la política
No tiene ningún mecanismo de alerta ante fallas	Una ejecución detenida o un resultado vacío pasan inadvertidos	Error Workflow para los fallos ruidosos y mensaje explícito de día sin movimientos para los silenciosos

Tabla 10 — Las tres carencias que impiden operar el flujo con volumen real

Además de esas tres hay una limitación de fondo que no se arregla agregando nodos. La política de negocio vive dentro de un nodo Switch, así que modificar un umbral obliga a entrar al editor de n8n, cambiar el flujo y volver a publicarlo. Con una política estable eso es aceptable, pero si los umbrales cambiaran cada mes, o si el área comercial necesitara ajustarlos sin depender de quien construyó el flujo, la política tendría que estar en una tabla parametrizable que el flujo consulte en cada ejecución.

Pasa algo parecido con la persona que atiende los casos derivados a revisión, que no tiene ninguna interfaz y tiene que ir a consultar la tabla de registro directamente. Con seis casos diarios es manejable, con ciento cincuenta deja de serlo.

La recomendación entonces es mantener el flujo operando de manera acotada, con el volumen actual de quince solicitudes diarias, mientras se incorporan la validación de entrada y el mecanismo de alerta, que son las dos correcciones de mayor impacto y menor costo. La parametrización de la política y la interfaz de revisión corresponden a una etapa posterior, cuando el volumen lo justifique. n8n no deja de servir cuando el volumen crece, deja de ser suficiente, y reconocer ese punto es criterio de ingeniería antes que una limitación de la herramienta.

Conclusiones

El laboratorio dejó ver que traducir una política de negocio a un flujo ejecutable es en buena medida un ejercicio de precisión sobre detalles que el enunciado no menciona explícitamente.

El orden de evaluación de las reglas resultó ser parte de la política y no una decisión de implementación. Dos solicitudes del conjunto de prueba cumplían varias reglas al mismo tiempo, y su clasificación dependió completamente de cuál se evaluaba primero.

Los tipos de dato resultaron ser una fuente de errores especialmente difícil de detectar, porque el dato correcto en el tipo equivocado sigue siendo un dato. La prueba que se hizo a propósito mostró además que un solo cambio puede producir al mismo tiempo una ruta equivocada y un motivo falso cuando la lógica está duplicada en dos lugares del flujo.

Por último, el conjunto de pruebas mostró que los fallos más costosos no son los que detienen el flujo sino los que lo dejan terminar correctamente produciendo un resultado incorrecto: el SKU inexistente aprobado, el filtro con un espacio invisible que reportaba cero aprobaciones, y el día sin solicitudes que es indistinguible de un flujo caído. Un sistema que nunca falla visiblemente no es un sistema confiable, y diseñar para que los errores sean ruidosos es preferible a esperar que no ocurran.